A Factory Method implementation still creates products (and in the case of PHP will still use the `new` keyword), but the act of creation is delegated to a single class method. Client code that was previously directly instantiating new objects directly now instead calls a creator to do the work for it. By doing this we are able to define a single interface that we are free to implement as we see fit as new products enter the design. Here is a class diagram showing how to implement the pattern:

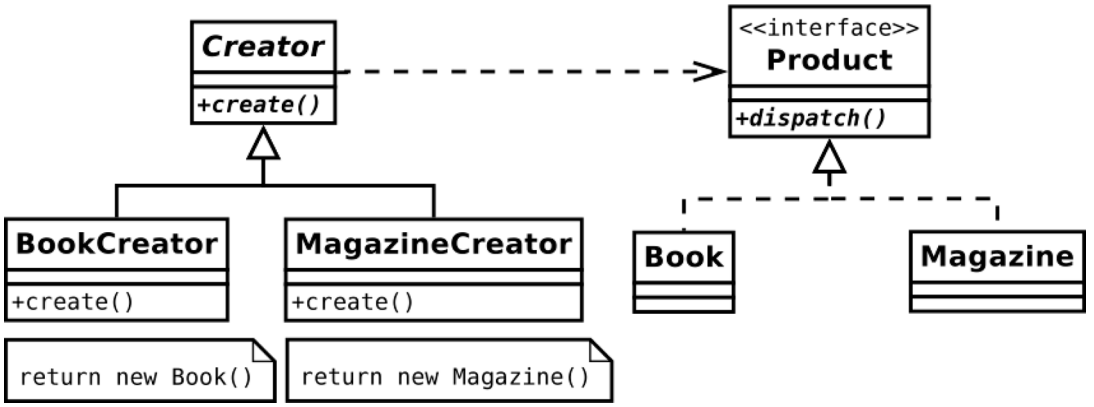


Figure 10-18. *Factory Method pattern*

The eagle-eyed among you might have spotted something familiar: the use of interfaces (`Product`) and abstract base classes (`Creator`) looks very much like the dependency inversion diagram in the previous section. This is of course exactly what is going on here, and you will encounter SOLID principles throughout design patterns. Depending on the specifics of your use-case, the `Creator` abstract class might not need to be a class at all: if the only reason for it to exist is to force inheriting subclasses to implement the `create()` method, it is probably better off being an interface too. A good pattern should offer some flexibility in the fine details and will serve as a framework for the overall design of your solution.

Moving on from Factory Method, a more complex problem space might introduce the need to create families of related objects. Our hypothetical print media retailer might branch out in the future and start selling, say, digital versions of the same media. We still need to represent `Book`, `Magazine`, and so on, but now we have an additional axis to model, resulting in `DigitalBook`, `PrintBook`, and `DigitalMagazine`. A more complex creational pattern exists for handling exactly this kind of complexity: the *Abstract Factory*. This pattern incorporates Factory Method but exchanges a single `create()` method for product-specific methods like `createBook()`, `createMagazine()`, and concrete factory classes are now specialized to the product family rather than the products: